

HandsMen Threads Salesforce CRM — Project Documentation

Salesforce Developer Project Documentation — HandsMen Threads

Elevating Men's Fashion with Salesforce

Project Title	HandsMen Threads – Salesforce CRM Implementation
Platform	Salesforce Developer Edition
Document Type	Project Documentation (Phase-wise)
Prepared By	Vikaas Anbarasan
Submission Date	September 2026

PROJECT OVERVIEW

For this project I designed and built a custom Salesforce CRM for HandsMen Threads, a men's fashion label that had started to outgrow its manual approach to tracking customers, orders and stock. Rather than maintaining separate spreadsheets for customers, products and orders, all of that information now lives inside a single Salesforce org, and a significant portion of the routine work — sending order confirmations, updating stock levels, moving a customer up a loyalty tier — now happens without anyone having to trigger it manually.

The build combines Salesforce's declarative tools (Custom Objects, Flows, Validation Rules) with a small amount of Apex where point-and-click configuration wasn't enough on its own, primarily for calculating order totals and updating stock and loyalty status in real time. The whole solution is housed within a single Lightning app, so the Customer, Order, Product, Inventory, Campaign and Reports tabs all sit together instead of being spread across the org.

OBJECTIVES

At the outset, the aim was straightforward: take the manual, error-prone aspects of running HandsMen Threads off someone's plate, and give the team a clearer, real-time view of what's happening across sales, stock and customers. That translated into a handful of concrete goals for the system.

- Automate repetitive work — order confirmation emails, loyalty tier updates, low-stock warnings — so nobody has to remember to carry them out manually.
- Prevent poor-quality data from entering the system in the first place, primarily through validation rules covering emails and stock levels.
- Give the sales, inventory and marketing functions visibility into stock levels, orders and customer activity without needing to check with one another.
- Ensure the different teams operate from a single shared source of data rather than their own separate records.
- Make the customer experience feel more personalised through the loyalty program and automated communications.

PHASE 1: REQUIREMENT ANALYSIS & PLANNING

Understanding Business Requirements

Before opening Salesforce, I spent time working through what was actually going wrong with the existing manual process. As HandsMen Threads' business grew, storing customer and order information in spreadsheets meant order totals had to be calculated by hand, there was no dependable way to spot stock running low, and customers received no confirmation once an order had been placed. What the business really needed was a single place to hold customer and product data, automatic calculation of order totals, a safeguard against stock going negative, and a way to keep customers informed without extra manual effort.

Defining Project Scope and Objectives

With that understanding in place, I scoped the project around a small number of core components rather than attempting to build everything at once:

- Custom objects for Customers, Products, Orders, Inventory and Marketing Campaigns, forming the backbone of the data model.
- Automation covering order confirmations, stock deduction and loyalty status updates — the three areas that were most burdensome to handle manually.
- Validation on customer email addresses and product stock so that poor-quality data cannot be saved in the first place.
- Separate profiles and roles for the Sales and Inventory functions, since their access needs differ.
- A single Lightning app bringing everything together, rather than a collection of disconnected tabs.

Data Model and Security Model

The data model rests on four custom objects, along with a separate Inventory object that tracks stock independently of the product record itself:

- HandsMen Customer (Customer__c) stores Name, Email, Phone, Address and Loyalty_Status__c.
- HandsMen Product (Product__c) stores Name, Description, Unit_Price__c and Stock__c.
- HandsMen Order (Order__c) stores the order date, Quantity__c and Total_Amount__c, and is linked back to both a Customer and a Product.
- Marketing Campaign (Marketing_Campaign__c) records promotions and marketing activity.
- Inventory tracks stock quantity by product and warehouse, and is the object that actually drives the low-stock alerts.

Security is handled across two layers. Profiles determine what a user is permitted to see and do overall — which objects, fields and pages they can access — while roles govern how much of that data they can see. For example, the Sales Manager role sits above the Sales Representative role in the hierarchy, so a manager can see the whole team's orders while a representative sees only their own.

PHASE 2: SALESFORCE DEVELOPMENT – BACKEND & CONFIGURATIONS

Setting Up the Environment

I registered for a free Salesforce Developer Edition org through developer.salesforce.com/signup, and carried out all of the configuration, testing and screenshots for this project within that org.

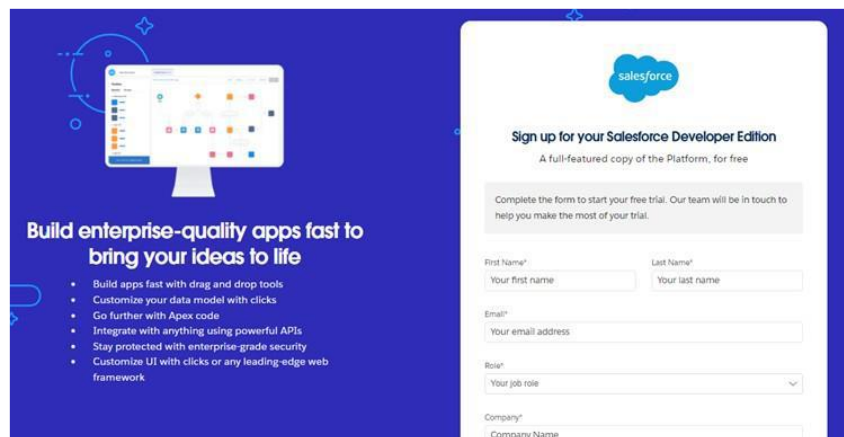


Fig 2.1: Salesforce Developer Edition Sign-Up

Objects, Fields, Validation Rules and Automation

The four custom objects identified in Phase 1 (Customer__c, Product__c, Order__c, Marketing_Campaign__c) were configured first, each with its own custom tab so that records could be created and browsed directly from the app rather than through Setup.

Data quality is enforced through two validation rules. The first sits on the product record and prevents Stock__c from ever being saved as a negative value — this is what stops an order from going through when it exceeds the stock available. The second sits on the customer record and requires the email field to include "@gmail.com" before the record can be saved, keeping customer contact details in a consistent format.

For automation, I built three separate Flows using Flow Builder instead of handling everything through Apex, since Flows are easier to revisit and adjust later:

- An order confirmation flow triggered whenever a HandsMen Order record is created, which emails the customer as soon as their order is placed.
- A low-stock alert flow, triggered on the Inventory object, which emails the inventory manager once a product's stock falls too low.
- A scheduled flow that runs once daily, iterates through every customer, checks their total purchases, and places them into the appropriate loyalty tier.

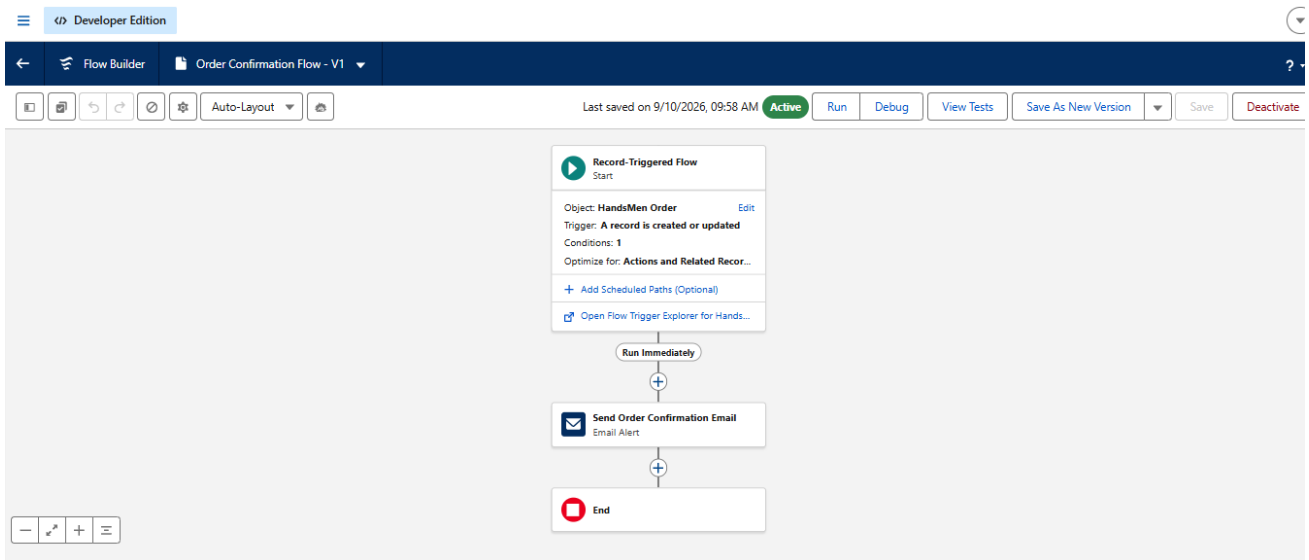


Fig 2.2: Order Confirmation Flow (Record-Triggered)

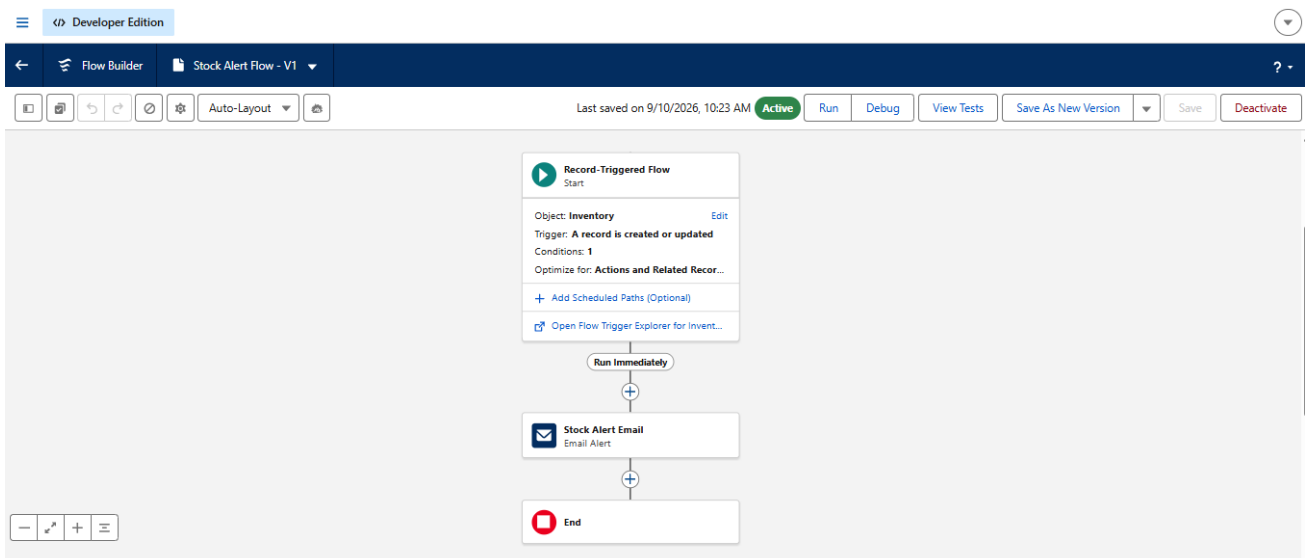


Fig 2.3: Low Stock Alert Flow (Record-Triggered on Inventory)

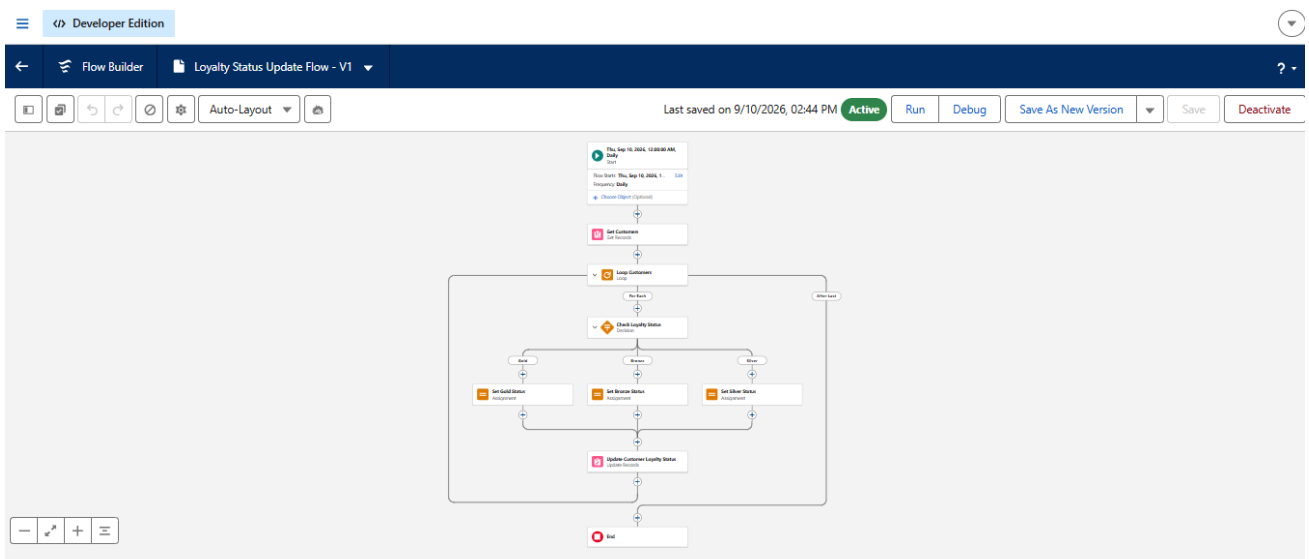


Fig 2.4: Scheduled Loyalty Status Update Flow (Daily, Schedule-Triggered)

Apex Classes & Triggers

A few pieces of logic couldn't be handled through clicks alone, so I implemented three Apex triggers to cover those. None of this logic needed to run asynchronously — there was no need for batch or queueable classes — since each trigger only performs a small amount of work at the point a record is saved.

- OrderTrigger calculates Total_Amount__c on an order using the quantity and the product's unit price, removing the need for anyone to work this out manually.
- ProductStockUpdateTrigger deducts the ordered quantity from the product's stock once an order is confirmed, relying on the stock validation rule to prevent stock from going negative.
- CustomerLoyaltyUpdateTrigger recalculates a customer's Loyalty_Status__c as soon as their purchase total changes, so they see the update immediately rather than waiting for the nightly scheduled flow.

PHASE 3: UI/UX DEVELOPMENT & CUSTOMIZATION

Building the App in App Manager

I assembled a Lightning app named "HandsMen Threads" using App Manager, bringing together the Customer, Orders, Products, Inventory, Reports and Dashboards tabs, so the team has a single app to open rather than searching for tabs individually.

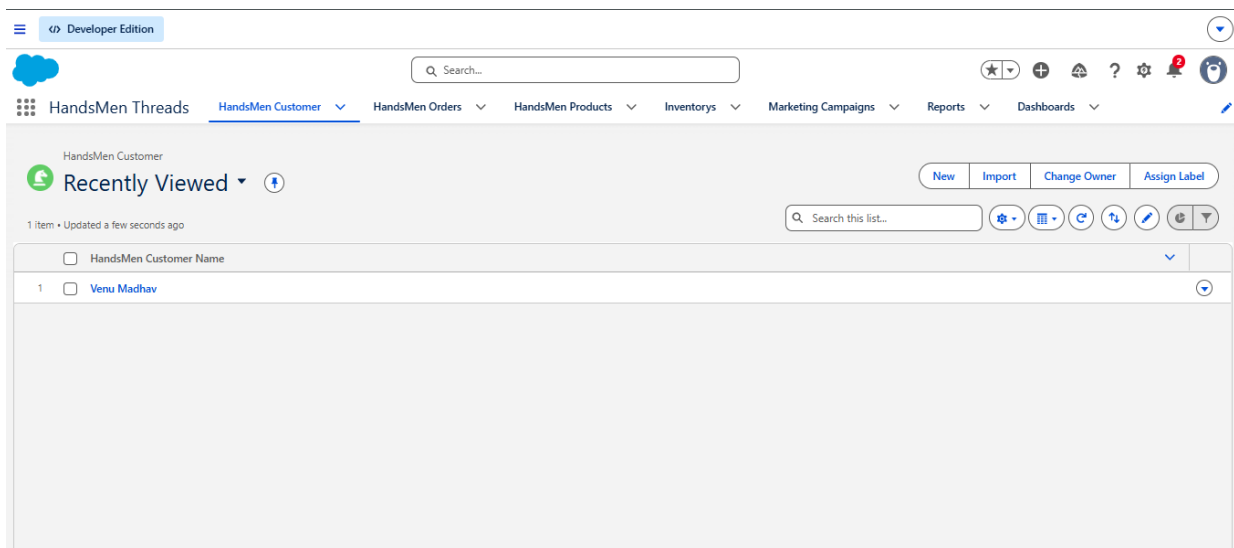


Fig 3.1: Custom "HandsMen Threads" Lightning App

Page Layouts & Record Creation

I kept the page layouts largely standard for each object, focusing mainly on arranging the fields so the most important ones — name, contact details, stock, order total — are visible without needing to scroll. A few of the record pages currently in use are shown below.

The screenshot shows the 'HandsMen Customer' record page for 'Venu Madhav'. The page has a top navigation bar with a search bar and various icons. Below the navigation bar, there's a sub-navigation bar with tabs for 'HandsMen Threads', 'HandsMen Customer', 'HandsMen Orders', 'HandsMen Products', 'Inventorys', 'Marketing Campaigns', 'Reports', and 'Dashboards'. The main content area is divided into two sections: 'Related' and 'Details'. The 'Details' section contains a form with fields for 'HandsMen Customer Name', 'Email', 'Phone', 'Loyalty Status', 'FirstName', 'LastName', 'Full_Name__c', and 'Total Purchases'. The 'Owner' field is set to 'Vikaas Anbarasan'. The 'Created By' field is set to 'Vikaas Anbarasan' with a timestamp of '9/10/2026, 7:58 AM'. The 'Last Modified By' field is also set to 'Vikaas Anbarasan' with a timestamp of '9/10/2026, 8:02 AM'.

Fig 3.2: Customer Record Creation (HandsMen Customer)

The screenshot shows the 'HandsMen Products' list view. The page has a top navigation bar with a search bar and various icons. Below the navigation bar, there's a sub-navigation bar with tabs for 'HandsMen Threads', 'HandsMen Customer', 'HandsMen Orders', 'HandsMen Products', 'Inventorys', 'Marketing Campaigns', 'Reports', and 'Dashboards'. The main content area is divided into two sections: 'Recently Viewed' and 'List View'. The 'List View' section contains a table with one item, 'Puma - Tshirt'. The table has columns for 'HandsMen Product Name' and 'Puma - Tshirt'. The 'Puma - Tshirt' row is highlighted. The table also has a search bar and a filter icon.

HandsMen Product Name
1 Puma - Tshirt

Fig 3.3: Product List View (HandsMen Products)



★
+
🏠
?
⚙️
🔔
👤

HandsMen Threads
HandsMen Customer
HandsMen Orders
HandsMen Products
Inventories
Marketing Campaigns
Reports
Dashboards


HandsMen Order
O-0001

New Contact
Edit
New Opportunity

Related
Details

HandsMen OrderNumber O-0001	Owner  Vikaas Anbarasan
HandsMen Customer Venu Madhav	
Status Confirmed	
Quantity 900	
Total Amount 550	
Customer Email vikaasanbarasan43@gmail.com	
Created By  Vikaas Anbarasan , 9/10/2026, 8:08 AM	Last Modified By  Vikaas Anbarasan , 9/10/2026, 8:08 AM

Fig 3.4: New Order Creation Page (HandsMen Orders)



★
+
🏠
?
⚙️
🔔
👤

HandsMen Threads
HandsMen Customer
HandsMen Orders
HandsMen Products
Inventories
Marketing Campaigns
Reports
Dashboards


Inventory
I -0001

New Contact
Edit
New Opportunity

Related
Details

Inventory Number I -0001	
HandsMen Product Puma - Tshirt	
Stock_Quantity	
Low Stock	
Stock_Status_c Low Stock	
Warehouse	
Created By  Vikaas Anbarasan , 9/10/2026, 8:10 AM	Last Modified By  Vikaas Anbarasan , 9/10/2026, 8:10 AM

Fig 3.5: Inventory Record

User Management

To confirm the multi-user setup functioned as intended, I created a couple of test users and assigned them to different profiles and roles — one configured for Sales, the other for Inventory — so that each user only sees the information relevant to their role.

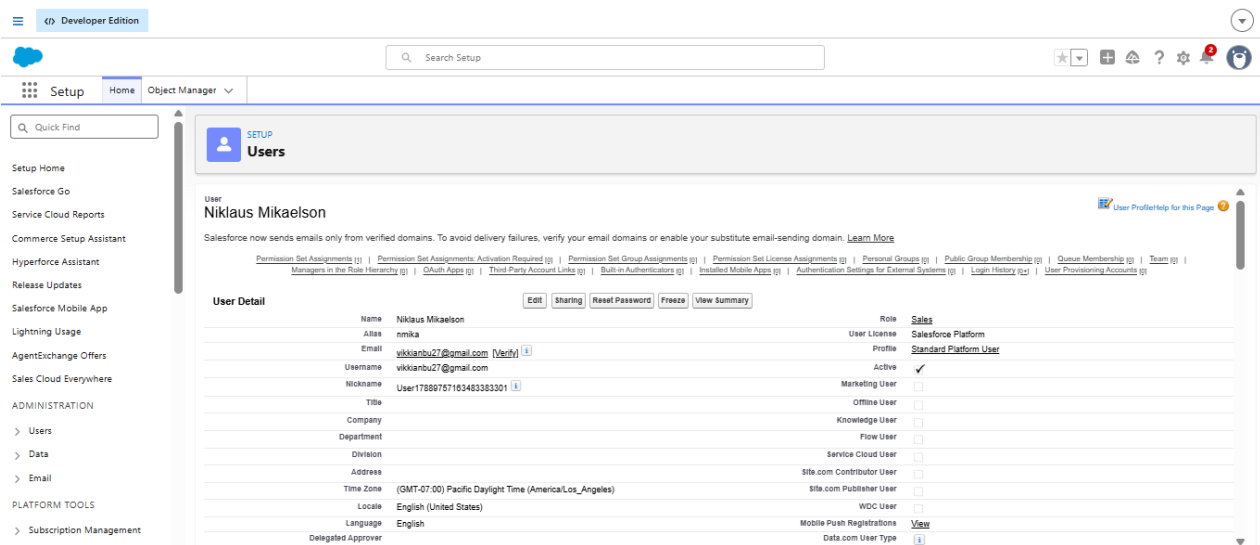


Fig 3.6: User Detail Page Showing Role and Profile Assignment

Reports and Dashboards

The app already has Reports and Dashboards tabs wired into it, providing a starting point for tracking sales and stock. Detailed dashboards haven't been built out yet — that is listed among the future enhancements at the end of this document.

PHASE 4: DATA MIGRATION, TESTING & SECURITY

Data Loading

For this build, sample Customer, Product, Order and Inventory records were entered directly through the app's UI, since the volume of data didn't justify anything more elaborate. If this system were rolled out using the brand's actual customer and product data, the Data Import Wizard would be suitable for a smaller, guided import, while Data Loader would be a better fit for a larger CSV-based migration.

Field History, Duplicate and Matching Rules

Loyalty_Status__c and Stock__c are the two fields most worth monitoring for changes, so enabling Field History Tracking on them would record who made a change and when — useful if a loyalty tier or stock figure ever appears incorrect and someone needs to trace back what happened. Duplicate and matching rules on the Customer object, matched against email and phone, prevent the same person from being added to the system more than once by mistake.

Profiles, Roles and Sharing

I cloned the Standard User profile to create a "HandsMen Sales Profile" with access tailored specifically to what a sales representative needs, rather than granting everyone the same broad set of permissions. Roles — Sales Manager, Inventory Manager, and others — are arranged in a hierarchy, so a manager automatically sees the records belonging to the people below them without anything needing to be explicitly shared. Where an individual requires slightly more access than their profile provides, without altering the profile for everyone else, a permission set is used instead. Sharing rules have been kept to a minimum for now, since the org-wide defaults cover most requirements,

though they would come into play if, for instance, the Inventory team needed visibility into Order records they currently cannot see.

Testing

Each of the three Apex triggers has a corresponding test class, covering both the scenario where everything works correctly (an order saves cleanly, with the total, stock and loyalty status all updating as expected) and the scenario where it shouldn't (attempting to oversell stock, or saving a customer record with an invalid email), confirming that the validation rules genuinely block bad data rather than merely appearing to.

Beyond the automated test classes, I also worked through the scenarios below manually within the org to verify that each piece of automation behaves correctly from end to end.

Feature	Test Scenario	Expected Result
Customer Email Validation Rule	Enter an email without "@gmail.com"	The save attempt is rejected, and an error message is shown
Order Total Trigger	Create an order for 1 unit priced at \$500	Total_Amount__c is calculated automatically as \$500
Email Notification Flow	Save a new confirmed order	The customer receives an order confirmation email
Stock Deduction Trigger	Confirm an order	Product Stock__c decreases by the quantity ordered
Product Stock Validation Rule	Try to order more units than are available in stock	The save is rejected, preventing an oversell
Low Stock Alert Flow	Inventory quantity falls below the set threshold	A low-stock alert email is sent to the Inventory Manager
Loyalty Status Trigger / Scheduled Flow	A customer's cumulative purchases cross a threshold	Loyalty_Status__c is updated (for example, to "Bronze")

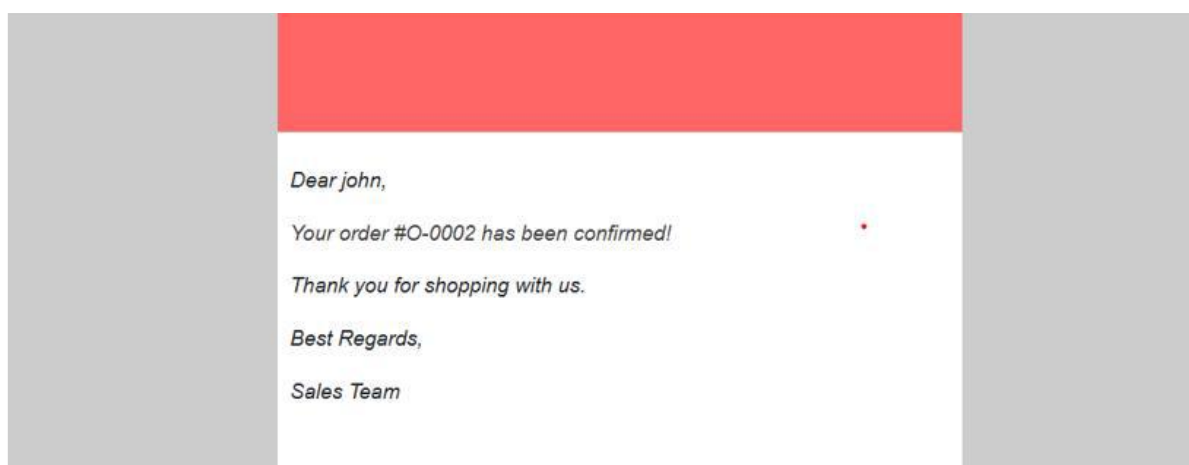


Fig 4.1: Order Confirmation Email — Output of Email Notification Flow

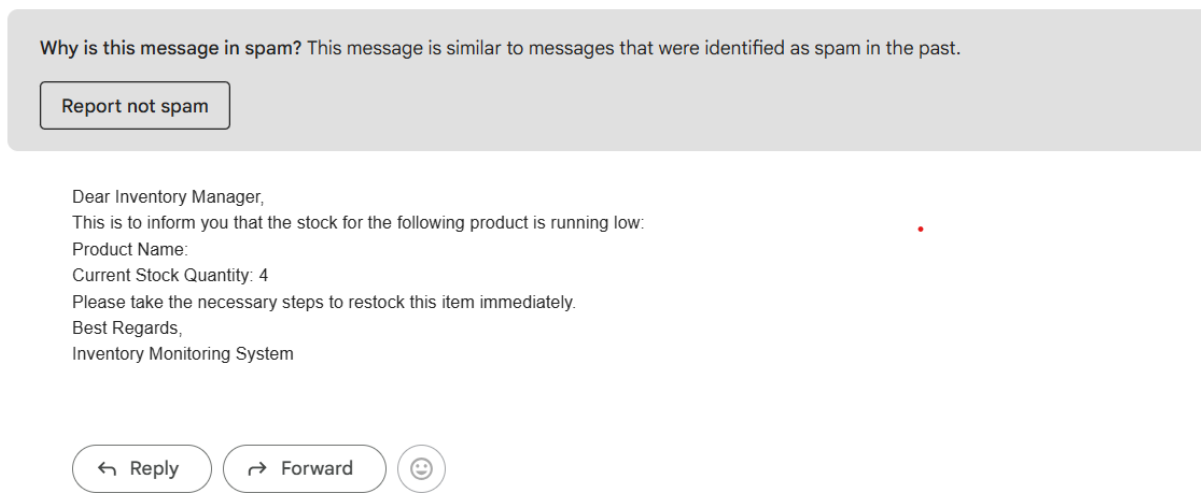


Fig 4.2: Low Stock Alert Email — Output of Stock Update Flow

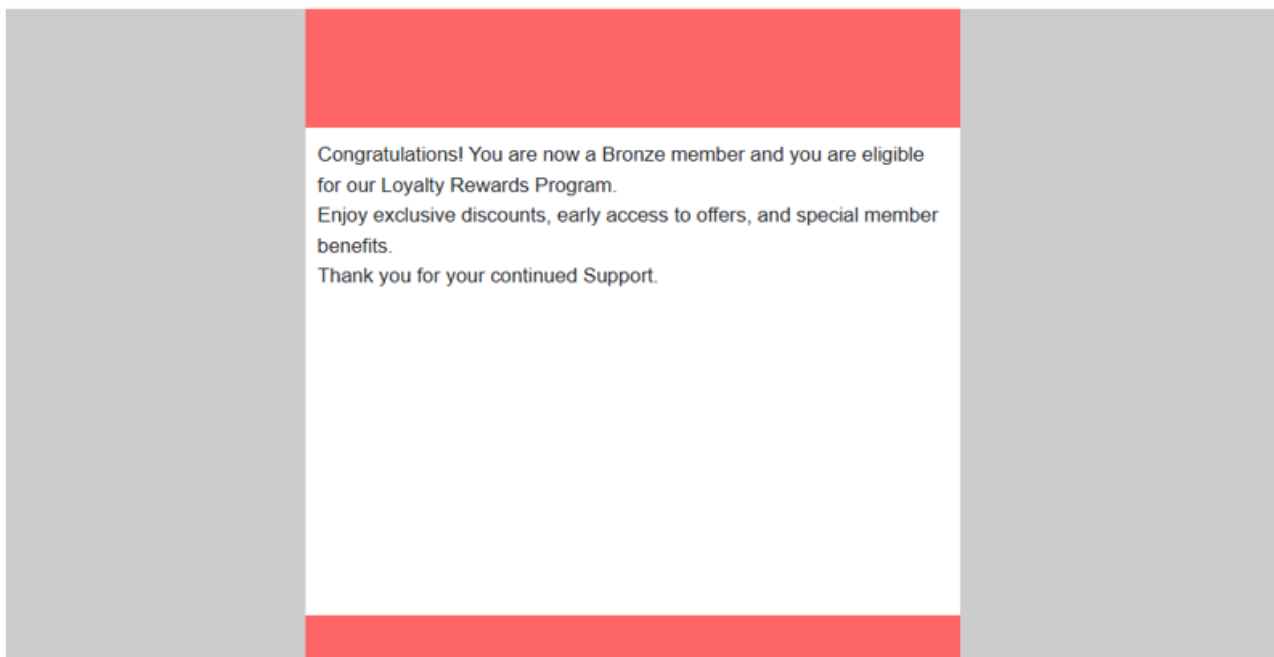


Fig 4.3: Loyalty Status Update Email — Output of Loyalty Status Trigger / Scheduled Flow

PHASE 5: DEPLOYMENT, DOCUMENTATION & MAINTENANCE

Deployment

The entire project was built and tested inside a single Developer Edition org, so there was no need to manage a deployment across separate environments. If this were to be promoted into an actual sandbox or production org, I would deploy it using change sets, moving components across in roughly dependency order — objects and fields first, followed by validation rules and flows, and finally the Apex classes and triggers along with their test coverage — validating each change set in the target org before committing it.

Maintenance & Monitoring

Going forward, keeping the system running smoothly mostly involves periodically checking the flows — reviewing run history to confirm the order confirmation, low-stock alert and loyalty flows are still firing as expected — and revisiting the validation rules if business requirements change, for instance if customers should be permitted to register with non-Gmail addresses. Field history on the key fields provides a way to investigate if something appears to have gone wrong.

Troubleshooting

When something fails to fire as expected, the first thing I check is the flow's run history, or the Apex debug logs if the issue relates to a trigger, simply to see at what point it stopped working. It's also worth checking whether a validation rule is silently blocking the save before assuming the automation itself is at fault. During testing, the low-stock alert email ended up in spam on a couple of occasions, which turned out to be a sender-deliverability setting rather than an issue with the flow itself — worth bearing in mind if alerts appear to go missing.

Conclusion

Overall, this project moved HandsMenThreads' customer, order, inventory and campaign data off spreadsheets and into a single Salesforce org, with the more tedious tasks — totalling orders, deducting stock, sending confirmations, updating loyalty tiers — now happening automatically rather than manually. There is still considerable scope to expand on it, but even at this stage it removes a meaningful amount of manual effort and offers a clearer picture of what's happening across sales and inventory than a collection of spreadsheets ever could.

Future Enhancements

A few directions I'd like to take this in if I continue developing it beyond its current scope:

- A self-service customer portal, allowing customers to log in and check their own past orders and loyalty points instead of having to ask.
- A mobile app built with the Salesforce Mobile SDK, aimed mainly at in-store staff who need to check stock or place an order without going to a desktop.
- Proper sales and inventory dashboards, since the Reports and Dashboards tabs are already set up but not yet built out.
- Einstein-based product recommendations, suggesting items to customers based on their previous purchases.
- WhatsApp or SMS integration for order and shipping updates, since not everyone checks email regularly.
- A basic chatbot to handle common customer questions without needing a person to respond to every one.